

Bow & Tie — Backend Developer Handoff

Project: Bow & Tie (a.k.a. BowClips) — boutique e-commerce store for handcrafted hair bows, clips & accessories **Market:** Dhaka, Bangladesh · Currency: **BDT** (৳) **Prepared:** 9 July 2026 · **Last updated:** 11 July 2026 **Audience:** Incoming backend developer — you will finish the backend, test the full project end-to-end, and integrate the remaining live third-party services (SSLCommerz payment gateway, WhatsApp customer messaging, real courier APIs).

What changed since 9 July (read this first): The app is now **deployed to a live staging environment** (Vercel + Render + Neon Postgres). Several things listed as "deferred" or "SMTP" in the first draft are **done**: product **variants as real SKUs**, **returns/refunds**, **review moderation**, **inventory view**, **email verification**, and **email now sends via Brevo's HTTP API** (Render blocks SMTP). The database is **PostgreSQL everywhere** (no more SQLite). Details throughout; deployment runbook is in **DEPLOY.md** at the repo root.

1. What this project is

A complete, working full-stack e-commerce application:

- **Storefront (customer-facing)** — browse catalog, search/filter, product galleries, **per-variant SKUs**, wishlist, cart, checkout, guest checkout, order history + tracking + **returns**, reviews with photos, product Q&A, **email verification**, **Google login**, newsletter, cross-sell ("You may also like").
- **Admin panel** (`/admin`) — dashboard, products (+ variants, + bulk import from CSV/Excel/PDF), **inventory (per-SKU stock)**, orders (+ printable invoices & courier shipping), **returns/refunds queue**, customers, promotions, coupons, sales reports, product Q&A moderation, **review moderation**, staff accounts with per-section permissions, settings.
- **API + database** — Node + Express + Prisma + **PostgreSQL**. All money math (prices, discounts, shipping, totals) is **recomputed server-side** — the client is never trusted.

Every third-party integration (email, WhatsApp, couriers, social login, payment) uses a **dev-fallback**: when credentials are absent it logs to the console / returns a mock, and goes live the moment you add the credentials. The remaining net-new build is **SSLCommerz** (\$7.1).

Using an AI coding assistant? The repo root has an **AGENTS.md** that Cursor, Claude Code, Copilot, Windsurf, etc. auto-load. If you paste project context into a chat AI, share the **Markdown** version of this document, not the PDF.

Tech stack

Layer	Technology
Backend runtime	Node.js (20+; developed on 24), TypeScript run via <code>tsx</code> (no build step)
Web framework	Express 4
ORM / DB	Prisma 6 — PostgreSQL everywhere (local: Neon or docker-compose; staging/prod: Neon)
Validation	Zod
Auth	JWT (<code>jsonwebtoken</code>) + <code>bcryptjs</code>
Uploads	Multer (product & review images → <code>backend/uploads/</code>) — ⚠️ ephemeral on Render, see §9
Email	Brevo HTTP API (primary) with Nodemailer SMTP fallback
Social login	<code>google-auth-library</code> (Google, live) + Facebook Graph API (wired, not configured)
Bulk import	<code>xlsx</code> (SheetJS) + <code>pdf-parse</code>
Frontend	React 19 + Vite + TypeScript, react-router-dom, Recharts (lazy)
State	React Context (StoreContext / AuthContext / ProductsContext)
Hosting	Frontend → Vercel · Backend → Render · DB → Neon (all free tier)

2. Current deployment (staging — LIVE)

Piece	Where	URL / notes
Storefront	Vercel	<code>https://bow-and-ties.vercel.app</code>
API	Render (free web service)	<code>https://bow-and-tie.onrender.com</code> — ⚠️ sleeps after ~15 min idle (~50s cold start)
Database	Neon PostgreSQL	Singapore region, free tier (0.5 GB)

Full step-by-step deploy/runbook: `DEPLOY.md` (repo root). Key facts a new dev must know:

- **Render blocks outbound SMTP ports** (25/465/587) → email uses the **Brevo HTTP API** instead (§7.3). Don't try to "fix" SMTP; it will always time out on Render.
- **Render's free disk is ephemeral** → uploaded images are wiped on every redeploy. Fine for staging; move to object storage before real launch (§9).
- **Render's free outbound IP can change** → it's allow-listed in Brevo's "Authorised IPs". If email suddenly 401s with an "unrecognised IP" message, add the new IP in Brevo → Security → Authorised IPs.
- **Auto-deploy:** both Render and Vercel auto-deploy on push to `master` of the `mashikurrahman/bow-and-tie` repo (no "s").
- ⚠️ **Repo gotcha:** a stray duplicate GitHub repo `bow-and-ties` (with an "s", default branch `main` , only an "Initial commit") exists and once caused Vercel to build stale code. Vercel is now correctly pointed at `bow-and-tie` (no "s"). The stray repo can be deleted.

- Build command on Render: `npm install && npm run deploy:db` (installs, `prisma generate` via postinstall, `prisma db push`, seed). Start: `npm start` (`tsx src/index.ts`).

3. Repository layout

```
Bow and Tie/
├── backend/                                # Node + Express + Prisma API
│   ├── prisma/
│   │   ├── schema.prisma                 # DB models (PostgreSQL)
│   │   ├── seed.ts                      # demo data loader (idempotent; env-driven admin)
│   │   └── seed-data.ts                 # product/promo catalog
│   ├── src/
│   │   ├── index.ts                    # entrypoint: connect DB, start server + schedulers
│   │   ├── app.ts                     # Express app + route mounting + CORS + /api/health
│   │   ├── config.ts                  # ALL env vars + business rules (shipping zones etc.)
│   │   ├── prisma.ts                 # Prisma client singleton
│   │   ├── middleware/
│   │   │   ├── auth.ts                # requireAuth/Admin/Staff, checkPermission, PERM_MAP
│   │   │   └── error.ts              # notFound + central error handler
│   │   ├── lib/
│   │   │   ├── mailer.ts              # Brevo HTTP API + SMTP fallback (deliver/sendMailResult)
│   │   │   ├── emails.ts             # HTML email templates (incl. verification)
│   │   │   ├── whatsapp.ts           # WhatsApp Cloud API alerts + weekly/monthly scheduler
│   │   │   ├── courier.ts            # Pathao / Steadfast / RedX consignments
│   │   │   ├── oauth.ts              # Google / Facebook token verification
│   │   │   ├── promotions.ts         # active-promotion pricing (skips variant products)
│   │   │   ├── inventory.ts          # variant-aware restock helper (cancel/refund)
│   │   │   ├── reviews.ts            # rating recompute from visible reviews
│   │   │   ├── importProducts.ts     # CSV/Excel/PDF parsing for bulk import
│   │   │   ├── abandonedCart.ts / stockAlerts.ts
│   │   │   ├── serialize.ts          # DB row → API JSON (hides costPrice; variant aggregates)
│   │   │   └── user.ts / auth.ts / ids.ts
│   │   └── routes/
│   │       ├── auth.ts               # register, login, oauth, forgot/reset, verify-email, resend, me
│   │       ├── products.ts           # catalog, variants, reviews, questions, related, notify-me
│   │       ├── orders.ts             # create, mine, track, cancel, return (variant-aware money/stock)
│   │       ├── account.ts / cart.ts / newsletter.ts / promotions.ts / upload.ts / sitemap.ts
│   │       └── admin.ts              # everything under /api/admin (see §6)
│   ├── uploads/                      # uploaded images (git-ignored, ephemeral on Render)
│   ├── .env.example                  # copy to .env and fill in
│   └── package.json
├── frontend/                          # Vite + React storefront + admin
│   ├── src/{pages,admin/pages,components,services,store}
│   ├── vercel.json                   # SPA rewrites
│   └── .env.example
├── proposal/                         # this document + client proposal/invoice
├── DEPLOY.md                         # free staging deploy runbook (Neon + Render + Vercel)
├── render.yaml                       # Render blueprint (backend)
├── docker-compose.yml                # optional local PostgreSQL
└── README.md
```

4. Running it locally (first 10 minutes)

The app runs on **PostgreSQL** now, so local dev needs a Postgres URL — easiest options:

- **Neon dev branch** (no install): create a branch in the Neon project and use its connection string, or
- **Local Docker:** `docker compose up -d` →
`DATABASE_URL="postgresql://postgres:postgres@localhost:5432/bowandtie?schema=public"`.

Backend

```
cd backend
npm install           # runs `prisma generate` via postinstall
cp .env.example .env  # set DATABASE_URL (Postgres) + JWT_SECRET
npm run deploy:db     # prisma db push + seed (schema + demo data)
npm run dev           # API → http://localhost:4000
```

Frontend

```
cd frontend
npm install
cp .env.example .env  # set VITE_API_URL=http://localhost:4000/api
npm run dev           # storefront → http://localhost:5173
```

Health check: `GET http://localhost:4000/api/health` → `{ "ok": true }`

Demo accounts (from the seed)

Role	Email	Password
Admin (dev default)	<code>admin@bowclips.com</code>	<code>admin123</code>
Customer (dev only)	<code>demo@bowclips.com</code>	<code>demo123</code>

On any public server, set `ADMIN_EMAIL` + `ADMIN_PASSWORD` — the seed then applies your strong password to the admin and **skips** the demo customer, so the site isn't protected by the well-known default login.

Windows note: Prisma's query-engine DLL locks while the dev server runs. If a Prisma command fails with `EPERM`, stop Node first (`taskkill /F /IM node.exe`), run the command, then restart `npm run dev`.

5. Data model (Prisma) — quick tour

Full schema: `backend/prisma/schema.prisma`. Conventions:

- **Money is stored as integers in BDT** (whole taka). No floats for currency.
- Arrays are stored as **JSON strings** (`colors`, `sizes`, `gallery`, `timeline`, `permissions`, review `images`, promotion `productIds`) and parsed by the serializers. (Now on Postgres you may optionally migrate these

to `Json / String[]` .)

- `costPrice` on `Product` is the buying cost — used for profit reports and **never exposed** by the public serializer.

Models: `User` (role customer/admin/staff, `permissions` , `emailVerified`), `Address` , `Product` , `ProductVariant` , `Review` (photo `images` , `hidden` for moderation), `Question` , `Order` + `OrderItem` , `Promo` (coupon codes), `Promotion` (campaigns/banners), `NewsletterSubscriber` , `AbandonedCart` , `StockAlert` .

`ProductVariant` (new — real SKUs): `label` , `color?` , `size?` , `price` , `stock` , `image?` , `sku?` , `sortOrder` . When a product has variants, its **display price = cheapest variant ("from")**, **stock = sum**, **inStock = any**; the order path uses the **selected variant's** price and decrements **that variant's** stock, then recomputes the aggregate. **Promotions do not stack on variant prices** (they're absolute per SKU). Products with no variants behave exactly as before.

`Order` carries: `deliveryZone` (inside/outside Dhaka), `payment` (cod/bkash/nagad), `txnId` , `courier` + `trackingCode` , `status` , JSON `timeline` , and **return/refund fields**: `returnReason` , `refundStatus` (`Requested` / `Approved` / `Rejected` / `Refunded`), `refundAmount` , `refundMethod` , `refundedAt` . `OrderItem` has `variantId?` so re-orders and restocks target the exact SKU.

6. Auth & permissions

- **JWT** in `Authorization: Bearer <token>` . Issued on register / login / oauth / verify-email.
- `requireAuth` — any logged-in user.
- `requireAdmin` — **reads role from the DB** (safe against stale tokens).
- `requireStaff` — allows `admin` OR `staff` ; attaches the user's `permissions` .
- `checkPermission` — maps request path → section key (`PERM_MAP` in `middleware/auth.ts`); **admins bypass, staff limited to granted sections**. Server-side, not just UI.

Admin sections (now): `dashboard` , `products` , `inventory` , `import` , `orders` , `returns` , `customers` , `promotions` , `coupons` , `reports` , `reviews` , `questions` , `settings` , `staff` . Staff can be granted any subset except `staff` .

Email verification (soft mode)

New customers are **signed in immediately** but flagged `emailVerified=false` , emailed a confirmation link, and shown a dismissible reminder banner until they verify. **Google-login users are auto-verified; admin/staff bypass**. Endpoints: `POST /auth/verify-email` (`{id, token}` → verifies + returns a session), `POST /auth/resend-verification` (auth). It's "soft" — nothing is blocked pre-verification (guest checkout already exists). Flip to hard-gating later if desired.

7. API surface (all under `/api`)

Auth — `routes/auth.ts` : `POST /auth/register` · `/auth/login` · `/auth/oauth` (`{provider, token}`) · `/auth/forgot-password` · `/auth/reset-password` · `/auth/verify-email` · `/auth/resend-verification` · `GET /auth/me` .

Products & content — `routes/products.ts` : `GET /products` · `GET /products/:id` · `GET /products/:id/related` · `GET/POST /products/:id/reviews` · `GET/POST /products/:id/questions` · `POST /products/:id/notify-me` . (Public review list and the product serializer **exclude hidden reviews**; rating is computed from visible reviews only.)

Orders — `routes/orders.ts` : `POST /orders` (**server recomputes** money; variant-aware price + stock; fires order email + WhatsApp alert) · `GET /orders/mine` · `GET /orders/:id` · `GET /orders/track/:id` · `POST /orders/:id/cancel` (variant-aware restock) · `POST /orders/:id/return` (sets `refundStatus=Requested`).

Account: `PUT /account/profile` , `POST /account/addresses` , `DELETE /account/addresses/:id` , `PUT /account/password` .

Other public: `POST /cart/track` , `POST /newsletter/subscribe` , `GET /promotions/active` , `GET /promotions/coupon/:code` (validate a checkout coupon against the live `Promo` table), `GET /promotions/:id` , `POST /upload` (auth), `POST /admin/upload` (admin).

Admin — `routes/admin.ts` (behind `requireAuth` → `requireStaff` → `checkPermission`):

- `GET /admin/stats`
- Orders: `GET /orders` , `GET /orders/:id` , `POST /orders/:id/ship` , `PATCH /orders/:id/status`
- **Returns/refunds:** `GET /returns` (queue), `PATCH /orders/:id/refund` (`{action: approve|reject|refund, amount?, method?}`) — refunding restocks items variant-aware)
- Products: `GET/POST /products` , `PUT/DELETE /products/:id` (accept a `variants` list), `POST /products/import` , `POST /products/import/commit`
- **Inventory:** `GET /inventory?threshold=` (per-SKU stock, low/out-of-stock summary)
- **Reviews:** `GET /reviews` , `PATCH /reviews/:id` (`{hidden}`), `DELETE /reviews/:id` (recompute rating)
- Q&A: `GET/PUT/DELETE /questions[/:id]`
- **Email diag:** `POST /email/test` (`{to?}`) → returns `{sent, error, via}`)
- WhatsApp: `POST /whatsapp/test` , `GET /whatsapp/report`
- `GET /reports` · `GET /customers` · promotions & coupons CRUD · staff CRUD

8. Third-party integrations — status & what you need to do

8.1 SSLCommerz payment gateway — ⚠ NOT YET INTEGRATED (your main build task)

Current state: Checkout accepts `payment: 'cod' | 'bkash' | 'nagad'` with an optional manual `txnId` . **No online gateway yet** — this is the primary integration to build.

What to build (SSLCommerz hosted checkout / EasyCheckout):

1. Add credentials to `config.ts` + `.env` : `SSLC_STORE_ID` , `SSLC_STORE_PASSWORD` , `SSLC_IS_LIVE` .
2. On order creation for an online payment, call the **Session API** with the server-computed `total` , `tran_id` = order id, and success/fail/cancel/IPN callback URLs. Redirect to the returned `GatewayPageURL` .
3. Add callback routes `POST /api/payments/sslcommerz/success|fail|cancel` + an **IPN** handler. On success, **validate server-side** via the Validation API (`val_id`) and confirm `amount` + `tran_id` match the order before marking paid — never trust the redirect alone.

4. Extend `Order` with payment-status fields (e.g. `paymentStatus`, `valid`, gateway response). Only mark paid/ `Confirmed` after IPN validation.
5. Keep COD working as-is.

Hook-in: `routes/orders.ts` `POST /orders` (money already computed there — branch on `payment`).

Frontend: `frontend/src/pages/CheckoutPage.tsx`. Staging gives you the required **public HTTPS callback URLs** (`https://bow-and-tie.onrender.com/...`). Use SSLCommerz **sandbox** first (`developer.sslcommerz.com`).

8.2 WhatsApp — admin alerts wired (needs creds); customer messages to build

Admin alerts (`lib/whatsapp.ts`, Meta Cloud API): new-order, low-stock, restock, and weekly/monthly sales reports. Without creds, logs to console. To go live: create a Meta app + WhatsApp Business account (needs a **Business portfolio** — currently blocked on access), get a **permanent** System User token (the dashboard token expires in 24h), set `WHATSAPP_TOKEN`, `WHATSAPP_PHONE_ID`, `WHATSAPP_ADMIN_TO` (intl format, no `+`). Test: `POST /api/admin/whatsapp/test`. The Meta **test number** can message up to 5 verified recipients — enough for staging.

Customer-facing WhatsApp (order confirmation/shipping updates to shoppers) is **not built** — Meta requires pre-approved **message templates** for business-initiated messages, plus opt-in. This is a planned follow-up; the sender/formatting pattern already exists to build on.

8.3 Transactional email — live via Brevo HTTP API

`lib/mailer.ts` + `lib/emails.ts`. Sends order confirmations/invoices, **email verification**, password reset, welcome, back-in-stock, abandoned-cart reminders.

Important: delivery uses **Brevo's HTTP API** (`https://api.brevo.com/v3/smtp/email`) via `BREVO_API_KEY` — because **Render blocks outbound SMTP**, nodemailer/SMTP times out there. SMTP (`SMTP_HOST` etc.) remains as a fallback for local/other hosts. Set `EMAIL_FROM` to a **Brevo-verified sender**. If sending 401s with an "unrecognised IP" message, add the server's outbound IP in **Brevo** → **Security** → **Authorised IPs**. Diagnose anytime with `POST /api/admin/email/test`.

8.4 Courier shipping — Steadfast live-ready; Pathao/RedX mock

`lib/courier.ts`. **Steadfast** has a real API call (activates with `STEADFAST_API_KEY` / `STEADFAST_SECRET`). **Pathao** and **RedX** return mock tracking codes — add their real API calls (marked `TODO`). Triggered from admin order page → `POST /admin/orders/:id/ship`.

8.5 Social login — Google live; Facebook wired

`lib/oauth.ts` verifies Google (ID token) and Facebook (access token) server-side. **Google is configured and working on staging** (`GOOGLE_CLIENT_ID` on backend, `VITE_GOOGLE_CLIENT_ID` on frontend; Vercel URL added to Google's Authorized JavaScript origins; app published). Facebook is coded but not set up (`FACEBOOK_APP_ID` / `SECRET` + `VITE_FACEBOOK_APP_ID`). Buttons hide until configured; Google users are auto-verified.

9. Environment variables

Backend (`backend/.env.example`):

Group	Vars
Core	<code>DATABASE_URL</code> (Postgres), <code>JWT_SECRET</code> , <code>JWT_EXPIRES_IN</code> , <code>PORT</code> , <code>NODE_ENV</code> , <code>CORS_ORIGIN</code> , <code>APP_URL</code> , <code>PUBLIC_URL</code> , <code>STORE_NAME</code>
Admin seed	<code>ADMIN_EMAIL</code> , <code>ADMIN_PASSWORD</code> (set on any public server)
Email	<code>BREVO_API_KEY</code> (primary), <code>EMAIL_FROM</code> , and SMTP fallback <code>SMTP_HOST/PORT/USER/PASS</code>
WhatsApp	<code>WHATSAPP_TOKEN</code> , <code>WHATSAPP_PHONE_ID</code> , <code>WHATSAPP_ADMIN_TO</code> , <code>LOW_STOCK_THRESHOLD</code>
Courier	<code>STEADFAST_API_KEY</code> , <code>STEADFAST_SECRET</code> , <code>PATHAO_CLIENT_ID</code> , <code>PATHAO_CLIENT_SECRET</code> , <code>REDX_API_TOKEN</code>
Social	<code>GOOGLE_CLIENT_ID</code> , <code>FACEBOOK_APP_ID</code> , <code>FACEBOOK_APP_SECRET</code>
Payment (to add)	<code>SSLC_STORE_ID</code> , <code>SSLC_STORE_PASSWORD</code> , <code>SSLC_IS_LIVE</code>

Frontend: `VITE_API_URL` (must end in `/api`), `VITE_GOOGLE_CLIENT_ID`, `VITE_FACEBOOK_APP_ID`.

10. Going to production (from staging)

Staging already covers most of this; for real launch:

1. **Database.** Already PostgreSQL (Neon). For prod, consider switching `db push` → real **migrations** (`prisma migrate`) for change history. Keep regular Neon backups.
2. **Secrets.** Strong `JWT_SECRET`; set `ADMIN_EMAIL` / `ADMIN_PASSWORD`; never commit `.env`.
3. **CORS.** Locked to `CORS_ORIGIN` in production (no localhost fallback) — set it to the real storefront origin.
4. **File uploads** → **object storage**. Images save to local `backend/uploads/`, which is **ephemeral on Render** (wiped on redeploy). Move to **Cloudflare R2 / S3** before loading the real catalog. (Contained change in `routes/upload.ts` + `PUBLIC_URL` /image URLs.)
5. **Email deliverability.** Move `EMAIL_FROM` from a Gmail freemail sender to a **verified custom domain** in Brevo (DKIM/DMARC) for inbox placement.
6. **Backend uptime.** Render free **sleeps when idle** — upgrade the instance (or keep-warm ping) so webhooks/IPN and alerts aren't delayed. A paid Render plan also gives a **static outbound IP** (removes the Brevo IP-allowlist chore).
7. **HTTPS callback URLs.** SSLCommerz IPN/callbacks and any WhatsApp webhook must be public HTTPS (staging URLs already are).

11. Suggested test plan (verify end-to-end)

Storefront: register → **verify email** (Brevo) → browse → open a product **with variants**, switch variants (price/stock/image update, sold-out disabled) → add to cart → apply a coupon (**create one in admin first** — e.g. a % code — and confirm it validates) → checkout (inside & outside Dhaka) → confirm the **server-**

computed total matches → order history → track → **request a return** → leave a review with a photo → ask a question. Also try **Google login**.

Admin: dashboard → create/edit a product **with variants** → **Inventory** page shows per-SKU stock → bulk import (CSV then PDF) → view an order → print invoice → ship with a courier → change status → **Returns queue:** approve → mark refunded (confirm stock is restored to the right variant) → **Reviews:** hide/unhide a review (confirm it drops from the storefront and rating recomputes) → answer a Q&A → sales report + CSV → create a **staff** account with limited permissions and verify it's **blocked** from other sections → create a coupon and a promotion.

Integrations (once creds added): SSLCommerz sandbox incl. IPN validation · a real WhatsApp alert · order-confirmation email (already live) · a real Steadfast consignment · Facebook login.

Regression watch-outs:

- All currency is integer BDT — no floats.
- Server stays the source of truth for pricing; don't move totals to the client.
- `costPrice` must never appear in public API responses.
- Staff permission checks stay server-side.
- Variant products: promotions must **not** stack on variant prices; cancel/refund must restock the **variant**, not just the product row.

12. Known gaps / roadmap after you

1. **SSLCommerz integration** — the one net-new build (\$8.1). Highest priority / launch blocker.
2. **Customer-facing WhatsApp** order updates (needs approved message templates); admin alerts already wired.
3. **Pathao / RedX** real courier API calls (Steadfast is done).
4. **Uploads** → **Cloudflare R2 / S3** (Render disk is ephemeral).
5. **Email domain auth** (custom-domain sender + DKIM/DMARC) for deliverability.
6. Optional: real Prisma **migrations** for prod; switch JSON-string columns to native Postgres `Json / String[]`.

Done since first draft (no longer gaps): product variants as real SKUs · returns/refunds · review moderation · admin inventory view · cross-sell/related products · email verification · Postgres migration · Google login · server-validated coupons.

13. Handy commands

```
# Backend
npm run dev           # start API (tsx watch)
npm run typecheck     # tsc --noEmit -- must stay clean
npm run deploy:db     # prisma db push + seed (schema + demo data)
npm run seed          # reseed only (idempotent)
npm run prisma:push    # apply schema to DB
npm run db:reset      # wipe + reseed

# Frontend
npm run dev           # Vite dev server
npm run build         # production build (must pass)
npm run lint          # oxlint
```

Contact / repo: code is in the private GitHub repo [mashikurrahman/bow-and-tie](#) (no "s"). Ask the owner for access. Start with the [README](#), then [DEPLOY.md](#), then this document, then [config.ts](#) and [routes/orders.ts](#).

Staging is live and everything except SSLCommerz runs today (with mocks/console fallbacks where creds are absent). Your work: build the SSLCommerz flow, add customer WhatsApp templates + the real Pathao/RedX calls, move uploads to object storage, and test the whole thing end-to-end.